

Methodology Report: Encounter-Level Neural Network Surrogate for SIMON

1. Purpose of this methodology

This methodology describes the encounter-level neural network developed as part of the Phase 2 SIMON work. The aim was not to replace the entire three-body integrator with a neural network. Instead, the aim was narrower and more controlled: to test whether a neural network can learn a local residual correction across a short close-encounter window.

The encounterNN is designed for Zone 3 encounters, where the pair separation is close enough that a fixed macro-step leapfrog update accumulates noticeable trajectory error, but not so close that Zone 2 adaptive sub-stepping dominates the physics. The tested prototype focuses on:

```
dt = 0.08 yr
active pair = bodies 1-2
encounter window = 0.5 yr
main deployment case = IC1, pair 1-2, strong approach
```

The central idea is:

```
Run a normal SIMON-noNN encounter window.
Ask the neural network to predict the residual between IAS15 and SIMON-noNN
at the window exit.
Add that predicted residual to the SIMON-noNN exit state.
Resume the long SIMON rollout.
```

Therefore, this is a residual-correction method, not a direct trajectory predictor and not a scalar force-correction model.

2. Background: why an encounter-level NN was considered

The Phase 2 review separated the close-encounter problem into two regions.

2.1 Zone 2

Zone 2 is the very close region below approximately 0.05 AU. The earlier method used a softened force and a scalar correction factor. However, the correction target in this region was analytically known:

```
c = (r_soft / r)^3
```

Multiplying the softened force by this correction simply restores the Newtonian force. Therefore, using a neural network in Zone 2 is methodologically unnecessary. The cleaner method is to use direct Newtonian force in Zone 2 and rely on adaptive sub-stepping for temporal resolution.

2.2 Zone 3

Zone 3 is the region from approximately 0.05 AU to 0.15 AU. In this region, softening is almost negligible, so the old analytic softening correction is close to 1. The remaining problem is not primarily force-law error; it is trajectory error from using a finite macro-step through a close approach.

This makes Zone 3 the more defensible region for learned correction. The scalar Zone-3 model corrects the force magnitude, while preserving analytic Newtonian force direction. The encounterNN work explored a more ambitious alternative: learning a whole encounter-window exit correction rather than a scalar force multiplier.

3. Method definition: what the encounterNN learns

The encounterNN learns a residual between two 0.5-year local window integrations.

For a given encounter start state I :

```
F_IAS = final state after IAS15 integration over 0.5 yr
F_S   = final state after SIMON-noNN integration over the same 0.5 yr
R     = F_IAS - F_S
```

The neural network is trained to learn:

```
NN(I) -> R_hat
```

At deployment:

```
F_corr = F_S,new + R_hat
```

where:

```
F_S,new = SIMON-noNN exit state for the new encounter window
R_hat   = neural network predicted residual
F_corr  = corrected encounter-exit state
```

This matches the flow diagram: the residual is the target, not part of the input. The model does not learn the scalar force correction c . It learns an 18-dimensional encounter-exit residual correction.

4. Input and output representation

4.1 Why the old scalar dataset was not enough

The old scalar Zone-3 dataset contained pair-level rows such as:

```
r_AU
r_soft
log_mi
log_mj
log_dt
v_rad_norm
v_tan_norm
c_opt
log_c_opt
```

That information is enough for a scalar force-correction model, because the model only predicts one number, c . However, it is not enough for an encounter-level residual model, because the encounterNN must predict corrections to the full positions and velocities of all three bodies.

Therefore, the encounterNN required newly generated full-state training data.

4.2 How the old scalar dataset was used

The file:

```
encounter_data_zone3_v3_augmented.npz
```

was present in the `encounter_nn` folder. The v1 and v2 generators could therefore load it automatically through the default argument:

```
--scalar-data encounter_data_zone3_v3_augmented.npz
```

This old scalar dataset was used as an empirical proposal distribution for pair-level quantities:

```
r_AU
log_mi
log_mj
v_rad_norm
v_tan_norm
```

This means the old scalar row was not used directly as a training example. Instead, it seeded the pair geometry and velocity.

For example, an old scalar row might say:

```
r_AU      = 0.12
log_mi    = log(0.01)
log_mj    = log(0.005)
v_rad_norm = -0.9
v_tan_norm = 0.8
```

The encounter generator uses this as a recipe:

```
Create a new full 3-body state where bodies 1 and 2 are separated by about
0.12 AU,
have masses about 0.01 and 0.005,
are approaching with v_rad_norm around -0.9,
and have tangential velocity around 0.8.
```

Then the generator adds the third body, chooses an orientation, constructs velocities, shifts to the centre-of-mass frame, runs IAS15, runs SIMON-noNN, and computes a new residual target.

So the old scalar row guides sampling, but the actual encounterNN sample is newly generated.

4.3 Final input vector: `X_rel18`

The final model uses a compact 18-dimensional relative-state input called `X_rel18`.

For the fixed active pair 1-2, this input contains:

log masses of the three bodies	3 values
relative position of body 2 with respect to body 1	3 values
relative velocity of body 2 with respect to body 1	3 values
third-body position relative to pair centre of mass	3 values
third-body velocity relative to pair centre of mass	3 values
log(dt), v_rad_norm, v_tan_norm	3 values

Total:

```
3 + 3 + 3 + 3 + 3 + 3 = 18 inputs
```

This representation is more physically meaningful than raw absolute coordinates because it focuses on relative geometry. It also removes unnecessary dependence on the absolute coordinate origin.

4.4 Output vector

The output is also 18-dimensional:

```
residual_x for 3 bodies = 9 values  
residual_v for 3 bodies = 9 values
```

So the model predicts:

```
[dx0, dy0, dz0, dx1, dy1, dz1, dx2, dy2, dz2,  
 dvx0, dvy0, dvz0, dvx1, dvy1, dvz1, dvx2, dvy2, dvz2]
```

This output is added to the SIMON-noNN encounter-window exit state.

5. Data-generation methodology

The data-generation process evolved in three stages.

5.1 v1 synthetic full-state generator

File:

```
generate_encounter_surrogate_data_v1.py
```

Purpose:

```
Create the first full-state encounter-window dataset.
```

For each accepted sample, the generator:

1. Samples a pair-level encounter condition using the old scalar Zone-3 dataset when available.
2. Builds a full 3-body planar state.
3. Shifts the state to the centre-of-mass frame.
4. Checks that the active pair starts in Zone 3.
5. Runs IAS15 over a 0.5-year window.
6. Runs revised SIMON-noNN over the same 0.5-year window.
7. Rejects invalid samples using physical and numerical filters.
8. Saves the residual:

```
residual_x = x_ias_final - x_nonn_final  
residual_v = v_ias_final - v_nonn_final
```

The v1 generator included quality filters for:

```
non-finite states
ejection
IAS15 energy drift
SIMON-noNN energy drift
minimum pair distance
Zone 3 start condition
residual outliers
```

It also stored rejection reasons so that failures could be diagnosed.

5.2 v2 event-targeted generator

File:

```
generate_encounter_surrogate_data_v2_event_targeted_fixed.py
```

Purpose:

```
Generate more data around the difficult real rollout event region.
```

After initial testing, the real encounter event occurred near:

```
t ≈ 82.56 yr
dt = 0.08
active pair = 1-2
r_pair ≈ 0.139 AU
v_rad_norm ≈ -1.1
v_tan_norm ≈ 0.85
```

The v2 generator kept the same full-state residual target as v1, but added a targeted mode:

```
event_strong_approach
```

This mode generated states with scalar pair features closer to the real event. It still produced full 3-body states and still ran IAS15 and SIMON-noNN to generate labels. The v2 data improved scalar-feature coverage, but later diagnostics showed that scalar-feature similarity was not enough. The actual event also needed to be close in the full `X_re118` state space.

5.3 v3 rollout-local generator

File:

```
generate_encounter_surrogate_data_v3_rollout_local.py
```

Purpose:

Generate training states close to the actual deployed event in full 18D state space.

This file was created because the synthetic v1/v2 data could match the real event in simple scalar features but still miss the actual full-state geometry.

The v3 generator works differently:

1. It runs the revised SIMON-noNN IC1 rollout.
2. It samples anchor states around the real event time, approximately $t = 82.56$ yr.
3. It adds small centre-of-mass-projected perturbations to the full position and velocity state.
4. It checks that pair 1-2 remains in Zone 3 and is approaching.
5. It runs IAS15 over the 0.5-year window.
6. It runs SIMON-noNN over the same window.
7. It saves the residual in the same format as v1/v2.

This makes v3 a targeted rollout-local enrichment dataset. It is not a broad generalisation dataset. It is a controlled local dataset designed to test whether the encounter residual map is learnable when the training distribution covers the deployed event geometry.

6. Model architecture

The final encounterNN model is a multi-layer perceptron.

Input:

18-dimensional X_{rel18}

Output:

18-dimensional residual vector

Architecture:

18 -> 128 -> 128 -> 128 -> 64 -> 18

Activation:

```
SiLU
```

Dropout:

```
optional, default 0.0 in the final usage
```

The model stores normalization buffers:

```
input_mean  
input_std  
target_mean  
target_std
```

During training, both input and target residuals are normalized. During inference, the raw `X_rel18` input is normalized, passed through the MLP, and then the predicted residual is unnormalized back into physical units.

This is important because position and velocity residuals have different natural scales. Without normalization, the network could overfit one part of the residual and ignore the other.

7. Training methodology

Main trainer:

```
train_encounter_surrogate_v2_velocitysafe_fixed2.py
```

The trainer uses the same scientific target as the original encounterNN design:

```
input  X = X_rel18  
target Y = [residual_x, residual_v]
```

The training process includes several safeguards.

7.1 Data validation

Before training, the loader checks that the dataset contains required fields:

```
X_rel18  
residual_x  
residual_v  
x_ias_final  
v_ias_final
```

```
x_nonn_final  
v_nonn_final  
m  
r_pair  
v_rad_norm  
v_tan_norm  
relE_ias  
relE_nonn  
pos_residual_rms  
vel_residual_rms  
category_id
```

It verifies that:

```
X_rel18 has shape (n, 18)  
residual_x has shape (n, 3, 3)  
residual_v has shape (n, 3, 3)  
all values are finite  
all masses are positive  
residual_x = x_ias_final - x_nonn_final  
residual_v = v_ias_final - v_nonn_final
```

This is a strong technical safeguard. It ensures the model is trained on the intended target, not on mislabeled or malformed data.

7.2 Stratified train/validation/test split

The trainer stratifies by encounter category:

```
strong approach  
weak/side  
recede  
broad
```

This prevents one category from disappearing from validation or test data.

7.3 Loss function

The final velocity-safe trainer uses a robust residual loss. The purpose is to improve both position and velocity safety.

The key change from the early trainer is that velocity is no longer given a very small weight. Earlier, the training placed much more emphasis on position. The velocity-safe trainer uses:

```
position weight = 1.0
velocity weight = 1.0
```

It uses SmoothL1/Huber-style loss on normalized residuals. This is appropriate because close-encounter residuals can have outliers; pure MSE would allow a few hard samples to dominate the training.

7.4 Physical consistency penalties

The trainer includes optional physical consistency penalties, especially for:

```
centre-of-mass position consistency
centre-of-mass velocity consistency
```

These discourage the network from adding artificial centre-of-mass drift. In the final training, these penalties were used with nonzero weights.

The trainer also includes an optional energy penalty, although the final setup did not rely on energy loss as the main constraint.

7.5 Checkpoint selection

The validation score does not look only at mean training loss. It combines:

```
median position error
median velocity error
position p95
velocity p95
unsafe position fraction
unsafe velocity fraction
```

This is technically sensible because the purpose is not only to fit average residuals. The model must avoid making many samples worse.

8. Inference and deployment methodology

The deployment path is:

1. Continue the global SIMON-noNN rollout.
2. At each macro-step, check whether the encounter gate fires.
3. If the gate does not fire, continue normally.
4. If the gate fires:
 5. run the local SIMON-noNN window,

6. construct `X_rel18` from the encounter entry state,
7. predict the residual using encounterNN,
8. compute the corrected exit state,
9. apply sanity checks,
10. resume the global rollout from the corrected state.

The correction formula is:

$$F_{\text{corr}} = F_{S,\text{new}} + R_{\text{hat}}$$

where:

```
F_S,new = SIMON-noNN 0.5-year window exit state  
R_hat   = encounterNN predicted residual
```

8.1 Gate condition

The final evaluation focuses on pair 1-2 and uses a strong-approach-style gate:

```
pair 1-2 in Zone 3  
v_rad_norm < threshold
```

A typical threshold used was:

```
v_rad_norm < -0.40
```

This means the encounterNN is not applied everywhere. It is only applied to the region where a meaningful correction is expected.

8.2 NumPy deployment

The optimized evaluator loads the PyTorch model once, extracts the linear-layer weights and normalization buffers into NumPy arrays, and performs the inference using NumPy.

This avoids calling PyTorch or CUDA inside the timed rollout. This is technically important because the encounterNN may fire only rarely; for a single small inference, GPU launch overhead can be larger than the computation itself.

8.3 Safety checks

The deployment includes several safeguards:

```
finite output check  
residual shape check
```

```
energy sanity gate
maximum radius/ejection gate
centre-of-mass projection
fallback to noNN if correction is invalid
```

The centre-of-mass projection is especially important because the network is allowed to predict a full 18D residual. Without projection or penalties, a small artificial centre-of-mass shift could be introduced.

9. Technical review of the neural network methodology

This section evaluates whether the encounterNN methodology is logically and technically sound, independent of whether the final results are strong.

9.1 Is the learning target logically correct?

Yes. The target:

$$R = \text{IAS15_exit} - \text{SIMON_noNN_exit}$$

is logically correct for a residual correction model.

The network is not asked to learn the entire future trajectory from scratch. It is only asked to learn the error made by SIMON-noNN over a bounded 0.5-year encounter window. This is a more stable and physically grounded target than directly predicting the IAS15 exit state.

At deployment, the formula:

$$F_{\text{corr}} = F_{S,\text{new}} + R_{\text{hat}}$$

is consistent with the training target.

Judgement:

Technically sound.

9.2 Is the input representation technically reasonable?

Yes, with limits.

The `X_re118` input is reasonable because it uses relative geometry rather than absolute coordinates. It includes:

```
masses
active-pair relative position
active-pair relative velocity
third-body state relative to the pair centre of mass
dt
radial velocity feature
tangential velocity feature
```

This is enough information for the fixed-pair, fixed-dt, planar IC1 prototype. It gives the network the main local ingredients that determine short-window encounter error.

However, the representation is not fully general. It is fixed to a particular active-pair ordering and does not have built-in permutation symmetry. If the method is later extended to arbitrary pairs or arbitrary initial conditions, the representation would need either:

```
pair-standardized ordering,
one model per active pair,
or a permutation-invariant architecture.
```

Judgement:

```
Technically sound for the current controlled prototype.
Not yet a general architecture for all three-body encounters.
```

9.3 Is the model architecture appropriate?

Yes.

A standard MLP is appropriate for this first residual-mapping experiment because the input and output are fixed-size vectors:

```
input: 18 values
output: 18 values
```

The hidden stack:

```
18 -> 128 -> 128 -> 128 -> 64 -> 18
```

has enough capacity to model nonlinear residuals without being excessively large. SiLU activations are suitable for smooth regression problems.

The architecture is not physically equivariant or symplectic. That is a limitation, but not a fatal flaw for a local residual prototype. The model is not being asked to replace the full integrator. It is correcting a bounded local window.

Judgement:

Technically acceptable for a local residual surrogate.
A more advanced architecture would be needed for broad generalisation.

9.4 Is the data generation technically valid?

Yes, provided it is described honestly.

The v1 and v2 synthetic generators are valid because they create full states, run IAS15 and noNN, and compute real residual targets. The old scalar dataset is only used to propose realistic pair-level variables.

The v3 rollout-local generator is also technically valid because it addresses a diagnosed distribution mismatch. It uses actual noNN rollout states near the deployed event and adds small perturbations. This is not cheating if it is presented as targeted local enrichment.

However, v3 should not be presented as evidence of broad generalisation. It is evidence that the residual correction is learnable when the training data covers the deployed local geometry.

Judgement:

Technically valid as a targeted local methodology.
Not sufficient to claim broad generalisation.

9.5 Is the training method technically sound?

Yes.

The trainer checks the dataset carefully, verifies residual labels, normalizes inputs and outputs, stratifies by encounter category, uses robust loss, and includes velocity safety and centre-of-mass consistency.

The move from a position-dominated loss to a velocity-safe loss is technically justified because correcting only position while harming velocity would be dynamically unsafe.

Judgement:

Technically sound.

9.6 Is the deployment method logically consistent?

Yes.

The deployment method matches the training method:

```
Training:  
R = IAS15_exit - noNN_exit
```

```
Deployment:  
corrected_exit = noNN_exit + predicted_R
```

The model is gated, not applied everywhere. That is technically correct because early experiments showed that a universal residual correction can harm easy weak/receding cases.

The evaluator also uses NumPy inference, which is the right deployment choice for a rarely fired small MLP.

Judgement:

Technically sound.

9.7 Main technical limitation

The main technical limitation is that the correction is a state-level jump. Even if the local exit state is more IAS15-like, a small state correction can shift the long-term chaotic branch. This is not a coding flaw. It is a known difficulty of chaotic systems.

The second limitation is that the model is not symplectic and does not exactly conserve energy or angular momentum. The methodology partly addresses this through:

```
physical data filters  
centre-of-mass penalties  
centre-of-mass projection  
fallback gates
```

but these do not make the neural correction fully structure-preserving.

The third limitation is scope. The final prototype is mainly valid for:

```
IC1  
dt = 0.08  
active pair 1-2  
0.5-year encounter window  
strong-approach deployment
```

It should not be claimed as a universal encounter surrogate.

10. Overall technical judgement

The encounterNN methodology is not technically flawed. It is logically coherent and internally consistent:

The target is well-defined.
The input representation is appropriate for the controlled prototype.
The output matches the residual-correction deployment formula.
The data generation produces true IAS15-vs-noNN residual labels.
The trainer validates data integrity and uses velocity-safe loss.
The evaluator applies the model with a gate and safety checks.

The main caution is about scope, not correctness. The method is technically valid as a local encounter-residual prototype. It should not be described as a general three-body neural integrator, a replacement for IAS15, a replacement for the scalar Zone-3 production model, or a universal skip-ahead simulator.

The most accurate technical conclusion is:

The encounterNN is a methodologically valid local residual-correction module. It tests whether a neural network can learn the IAS15-minus-noNN exit-state error over a bounded close-encounter window. The implementation is technically coherent, but its current design is local, gated, fixed-pair, fixed-dt, and not yet a general-purpose neural integrator.

11. Mentor-facing summary

The neural network methodology is technically defensible because it does not ask the NN to replace Newtonian dynamics. It keeps SIMON as the physical backbone and trains the network only to predict a local residual over a short encounter window. The training labels are generated from fresh IAS15 and SIMON-noNN simulations, not copied from the old scalar dataset. The old scalar dataset only supplies realistic pair-level sampling variables for the early synthetic generators.

The model uses a compact 18-dimensional relative state, predicts an 18-dimensional residual, and applies the correction only through:

```
corrected_exit = SIMON_noNN_exit + predicted_residual
```

The methodology includes quality filters, data validation, normalization, velocity-safe training, centre-of-mass safeguards, gated deployment, and fallback checks. Therefore, the design is not technically flawed. Its limitation is that it is a local prototype and should be presented as such.